

CS339: Abstractions and Paradigms for Programming

Higher Order List Functions

Manas Thakur
CSE, IIT Bombay



Autumn 2026

1. Repetitions galore

- Recall the program to sum a list:

```
(define (sum l)
  (if (null? l)
      0
      (+ (car l) (sum (cdr l)))))
```

- What about product?

```
(define (prod l)
  (if (null? l)
      1
      (* (car l) (prod (cdr l)))))
```

- Length?

```
(define (length l)
  (if (null? l)
      0
      (+ 1 (length (cdr l)))))
```

Along with similar syntax, what are we doing in each of these?

Folding a list to a single value!



The fold higher-order operation

```
(define (foldr f v l)
  (if (null? l)
      v
      (f (car l) (foldr f v (cdr l)))))
```

Also called **reduce**
or **accumulate**.

➤ Updated sum:

```
(define (sum l)
  (foldr + 0 l))
```

➤ Length?

```
(define (length l)
  (foldr (lambda (x y) (+ 1 y))
        0 l))
```

➤ Product:

```
(define (prod l)
  (foldr * 1 l))
```



What was the 'r' in "foldr"?

```
(define (foldr f v l)
  (if (null? l)
      v
      (f (car l) (foldr f v (cdr l)))))
```

Folds from *right*.

```
> (define l (list 1 2 3 4))
> (foldr + 0 l)
```

```
(+ 1 (+ 2 (+ 3 (+ 4 0))))
```

► We also have a foldl:

```
(define (foldl f v l)
  (if (null? l)
      v
      (foldl f (f v (car l)) (cdr l))))
```

Folds from *left*.

```
> (foldl + 0 l)
```

```
(+ 0 1) ==> (+ 1 2) ==>
(+ 3 3) ==> (+ 6 4) ==> 10
```



2. Transforming lists

- Add 10 to each element of a list:

```
(define (add10 l)
  (if (null? l)
      nil
      (cons (+ 10 (car l)) (add10 (cdr l)))))
```

We are *mapping* a list to another list by applying a common function to all its elements!

- How about squaring each element?

```
(define (sqr-list l)
  (if (null? l)
      nil
      (cons (square (car l)) (sqr-list (cdr l)))))
```



The **map** higher-order operation

```
(define (map f l)
  (if (null? l)
      nil
      (cons (f (car l)) (map f (cdr l)))))
```

➤ What's squaring a list now?

```
(define (sqr-list l)
  (map square l))
```

➤ The new add10 function:

```
(define (add10 l)
  (map (lambda (x) (+ x 10)) l))
```

Map-based functions usually are also great candidates for parallelization!

Something complex (for now!)

- Compute the sum of the squares of the even numbers of a list:

```
(define (sum-sqr-evn l)
  (if (null? l)
      0
      (let ((x (car l)))
        (if (even? x)
            (+ (square x) (sum-sqr-evn (cdr l)))
            (sum-sqr-evn (cdr l))))))
```

Now keep this aside for some time.



3. Filtering lists

- Return numbers from a list that are >10 :

```
(define (gt10 l)
  (if (null? l)
      nil
      (if (> (car l) 10)
          (cons (car l) (gt10 (cdr l)))
          (gt10 (cdr l)))))
```

You know what we're going to do next!

- Return even numbers from a list:

```
(define (get-evn l)
  (if (null? l)
      nil
      (if (even? (car l))
          (cons (car l) (get-evn (cdr l)))
          (get-evn (cdr l)))))
```



The **filter** higher-order operation

```
(define (filter pred l)
  (if (null? l)
      nil
      (let ((x (car l)))
        (if (pred x)
            (cons x (filter pred (cdr l)))
            (filter pred (cdr l)))))))
```

➤ The new get-evn:

```
(define (get-evn l)
  (filter even? l))
```

➤ The new gt10:

```
(define (gt10 l)
  (filter (lambda (x) (> x 10)) l))
```

Now we can solve complex problems so elegantly!

- Get a list containing squares of the even numbers of a list:

```
(define (sqr-evn l)
  (map square (filter even? l)))
```

- Compute the sum of the squares of the even numbers of a list:

```
(define (sum-sqr-evn l)
  (foldr + 0 (map square (filter even? l))))
```

Once again notice how the new vocabulary created a new abstraction, and how it led to a significant amount of program simplification.

```
(define (sum-sqr-evn l)
  (if (null? l)
      0
      (let ((x (car l)))
        (if (even? x)
            (+ (square x) (sum-sqr-evn (cdr l)))
            (sum-sqr-evn (cdr l))))))
```

Comparison



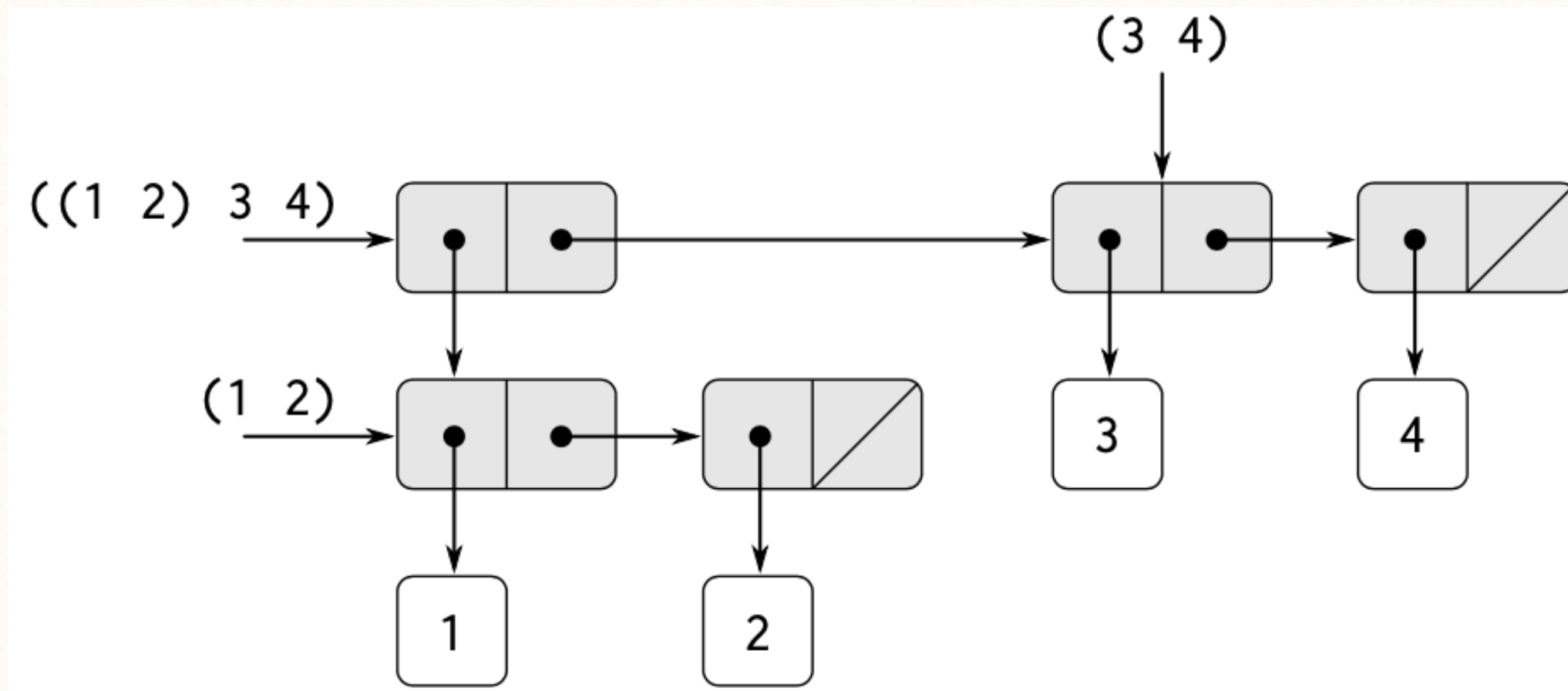
Sequences as Conventional Interfaces



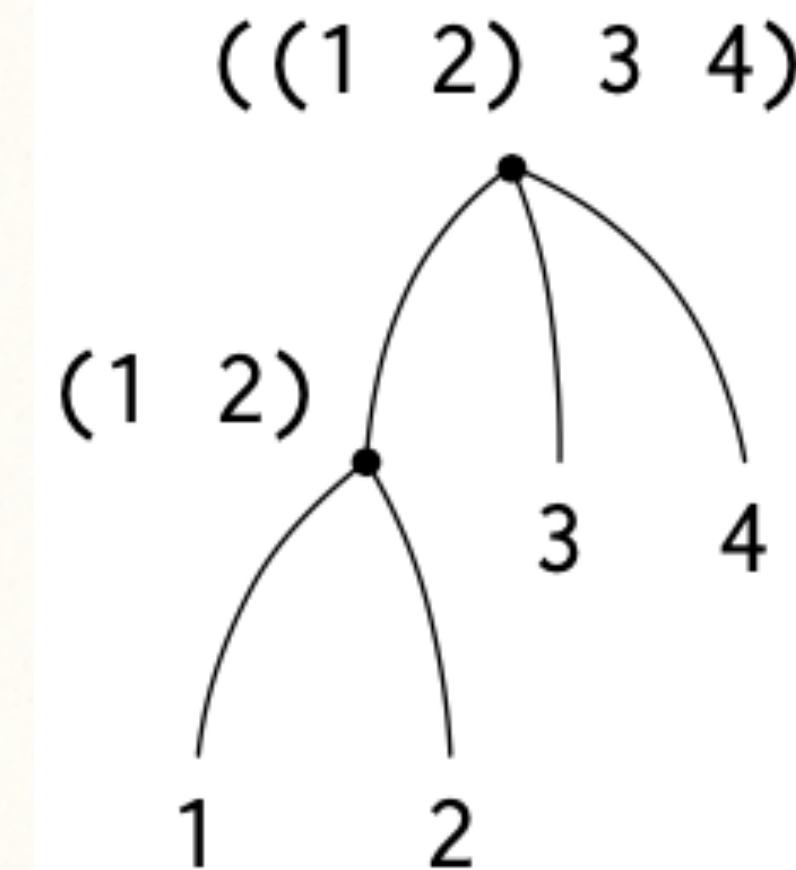
Let's get back to lists

- Our lists look very similar to trees:

`(list (list 1 2) 3 4)`



- What's the corresponding tree?



Sum the squares of the odd leaves of a tree

```
(define (sum-odd-squares tree)
  (cond ((null? tree) 0)
        ((not (pair? tree))
         (if (odd? tree) (square tree) 0))
        (else (+ (sum-odd-squares (car tree))
                   (sum-odd-squares (cdr tree))))))
```

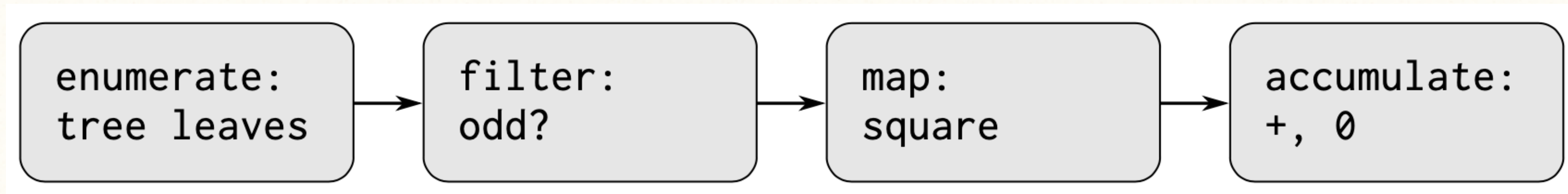
List the even fibonacci numbers till $fib(n)$

```
(define (even-fibs n)
  (define (next k)
    (if (> k n)
        nil
        (let ((f (fib k)))
          (if (even? f)
              (cons f (next (+ k 1)))
              (next (+ k 1))))))
  (next 0))
```

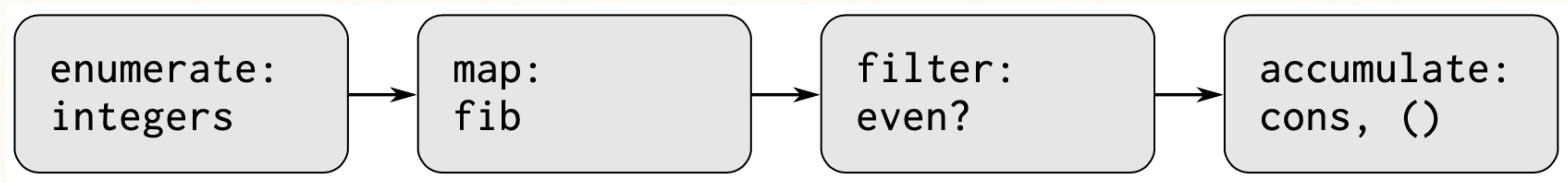
Is there any similarity between these two programs?



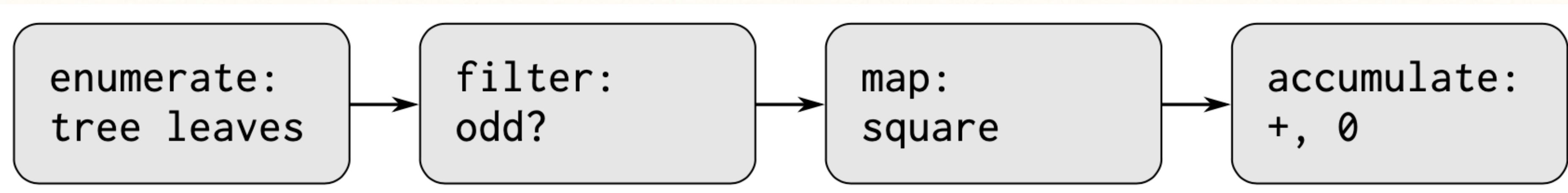
Sum the squares of the odd leaves of a tree



List the even fibonacci numbers till $fib(n)$



Sum the squares of the odd leaves of a tree

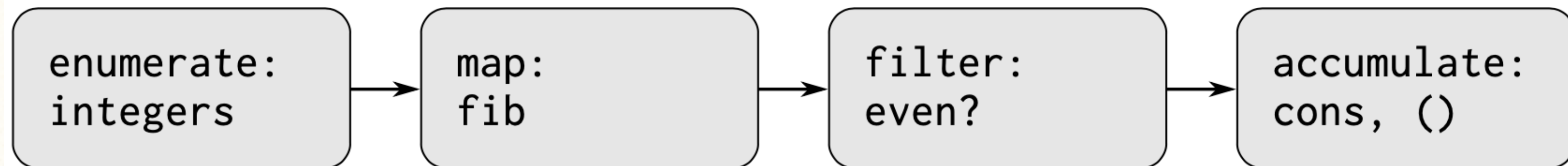


```
(define (sum-odd-squares tree)
  (cond ((null? tree) 0)
        ((not (pair? tree))
         (if (odd? tree) (square tree) 0))
        (else (+ (sum-odd-squares (car tree))
                   (sum-odd-squares (cdr tree))))))
```

Don't match!

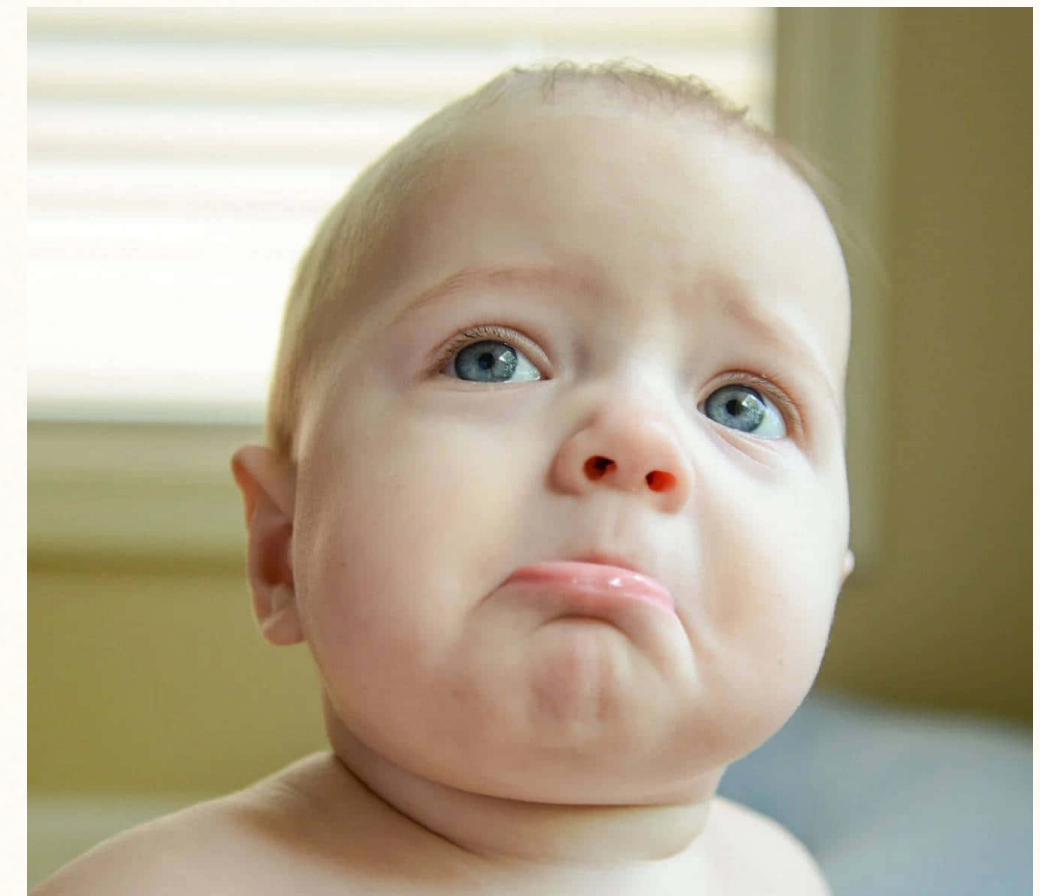


List the even fibonacci numbers till $fib(n)$



```
(define (even-fibs n)
  (define (next k)
    (if (> k n)
        nil
        (let ((f (fib k)))
          (if (even? f)
              (cons f (next (+ k 1)))
              (next (+ k 1))))))
  (next 0))
```

Don't match again!



Enumerators

```
(define (enumerate-interval low high)
  (if (> low high)
      '()
      (cons low (enumerate-interval (+ low 1) high))))

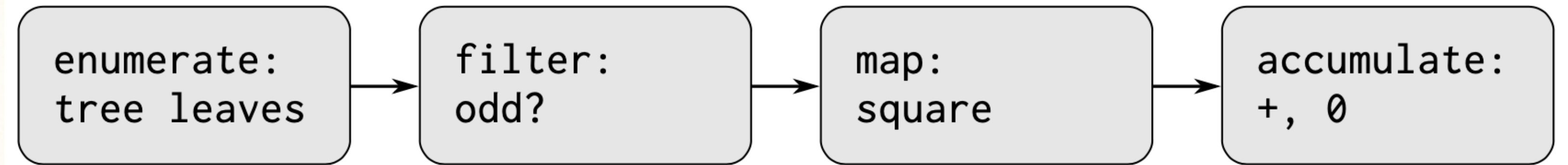
> (enumerate-interval 1 5)
> (1 2 3 4 5)
```

```
(define (enumerate-tree tree)
  (cond ((null? tree) '())
        ((not (pair? tree)) (list tree))
        (else (append (enumerate-tree (car tree))
                        (enumerate-tree (cdr tree))))))

> (enumerate-tree (list 1 (list 2 (list 3 4)) 5))
> (1 2 3 4 5)
```



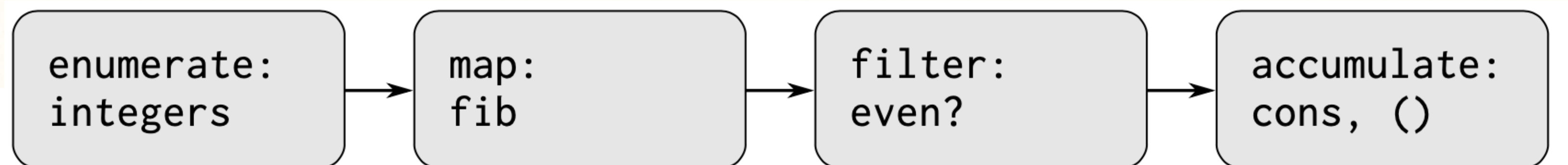
Sum the squares of the odd leaves of a tree



```
(define (sum-odd-squares tree)
  (accumulate
    + 0 (map square (filter odd? (enumerate-tree tree)))))
```



List the even fibonacci numbers till $fib(n)$



```
(define (even-fibs n)
  (accumulate
    cons
    nil
    (filter even? (map fib (enumerate-interval 0 n)))))
```

A PL for specifying computations using sequences

- What constitutes a programming language?
 - Primitive expressions; means of combination; means of abstraction
- We have created a new **programming language** that defines *sequences as conventional interfaces*:
 - Sequences (implemented using lists) are like signals flowing from one stage to another
 - `map`, `filter` and `accumulate` (the new vocabulary of our language) represent general patterns of processing sequences
 - **Advantage:** Largely decoupled **modules** defined in terms of operations on sequences
 - Key to reducing complexity of large software



Unevaluated...



Next class, we abandon 40 years of functions-first thinking and ask what happens if data fights back.